

ACEMID_Interim_Study_Data_v2

July 17, 2025

1 ACEMID Interim Study Data

1.1 Prerequisites

- Access to a Windows PC that can connect to the Vectra's database and storage drive.
- 500GB to 1TB of secure storage for the copied files and processed data.
- A list of the study IDs (as recorded in the Vectra database) to analyse.
- Python 3.8+
 - Jupyter
 - Pandas
 - tqdm
- VectraDBTool_v1.8.exe
- surfcopy.exe

1.2 Instructions

1.2.1 Step 1 - Python setup

Import the Python packages required by this notebook. Any issues with missing packages must be addressed before proceeding. If you get the `ModuleNotFoundError: No module named 'module_name'` you will need to install the missing module, for example using pip: `pip install module_name`.

```
[ ]: import pandas as pd
      from pathlib import Path
      import os
      import subprocess
      import shutil
      from tqdm import tqdm
```

1.2.2 Step 2 - VectraDBTool Setup

Copy the supplied **VectraDBTool_v1.8.exe** file into the same folder as the Vectra software (**vectra.exe**). In many cases, this folder will be located at: `C:\Vectra\bin\` (and therefore the VectraDBTool will end up at `C:\vectra\bin\VectraDBTool_v1.8.exe`).

1.2.3 Step 3 - Load study IDs

Set the current site, so that the script only loads study IDs from that site. If your Vectra database holds data for multiple sites, the data for these sites can be loaded simultaneously.

Site List

- 1) QLD - PAH
- 2) QLD - HIRF (RBWH)
- 3) NSW - MIA
- 4) NSW - Westmead
- 5) VIC - Alfred Health
- 6) VIC - Skin Health Institute
- 7) VIC - Bendigo

Set your site number(s) in the `site_number` variable of the next cell (line 1). If you are loading data from multiple sites, enter the site numbers as comma-separated values.

Single site example:

```
site_number: list[int] = [1]
```

Multi-site example:

```
site_number: list[int] = [5,6,7]
```

```
[ ]: site_number: list[int] = [1]

# Don't change the rest of this cell

site_list: dict = {
    1: "QLD 1 - PAH",
    2: "QLD 4 - HIRF (RBWH)",
    3: "NSW 3 - MIA",
    4: "NSW 1 - Westmead",
    5: "VIC 1 - Alfred Health",
    6: "VIC 2 - Skin Health Institute",
    7: "VIC 4 - Bendigo Health",
}

current_sites: list[str] = [site_list[n] for n in site_number]

interim_ids_all_sites: pd.DataFrame = pd.read_csv(
    "ACEMID-24m participants - Melanomas 29May25.xlsx - Overall.csv"
)
interim_ids_all_sites["Site ID"] = interim_ids_all_sites["Site ID"].str.strip()
interim_ids_all_sites.columns = interim_ids_all_sites.columns.str.strip()
interim_ids_current_site: pd.DataFrame =
    ↪ interim_ids_all_sites[interim_ids_all_sites["Site ID"].isin(current_sites)]

print(f"Loaded {len(interim_ids_current_site)} study IDs from
    ↪ {len(current_sites)} site(s).")
```

Run the next cell to view the ID list (for manual verification etc)

```
[ ]: interim_ids_current_site["Study ID"]
```

1.2.4 Step 4 - Export patient data

During this step, we will perform a cleansed data export of the participant IDs selected during Step 2. In this context, “cleansed” refers only to removal of patient metadata (name, DOB, contact details) - all 2D and 3D imagery will remain intact.

You will need to set the following variables (*note: use forward-slashes rather than back-slashes in the `exe_path` and `exportdir` values, e.g. `e:/output/folder/`*)

- **exe_path**: The path (location) of the **VectraDBTool_v1.8.exe** file (see **Step 2**).
- **exportdir**: The destination folder for the exported data.
- **dbname**: The name of the Vectra database containing the ACEMID participants. If you don't know the database name, run the **VectraDBTool_1.8.exe** program; when the program starts, it will present a list of known databases in a dropdown box under the heading **Select a database**.

```
[ ]: exe_path: str = "c:/vectra/bin/VectraDBTool_v1.8.exe"
      exportdir: str = "e:/exported/cleansed/"
      dbname: str = "VectraDatabase"
```

Running the next cell will perform the export. This process takes place in the background, and there will not be any feedback or status updates during the process. Monitor the export destination folder to check the status of the export (note that each exported study ID will produce 1 x file and 1 x folder in the destination folder).

If you encounter the error: **Failed to initialize the database (database name)**, it is likely that you have not supplied the correct database name. Note that the database name is **case-sensitive** and may contain spaces.

```
[ ]: exportids: str = ",".join(interim_ids_current_site["Study ID"].to_list())

      export_args = [
          exe_path,
          "-silentexport",
          "-exportdir",
          exportdir,
          "-exportids",
          exportids,
          "-cleanse",
          "-exportimages",
          "-dbname",
          dbname
      ]

      subprocess.run(export_args)
```

1.2.5 Step 5 - Verify exported data

Once the export is complete, we will verify that the destination folder contains the expected number of files and folders. As previously mentioned, each exported study ID will produce 1 x `.db` file and

1 x folder in the destination folder. The following cell will count the number of files/folders in the destination folder and compare this to the number of study IDs supplied to the export program.

```
[ ]: exported_file_count = [x for x in Path(exportdir).glob("*.db") if x.is_file()]
exported_folder_count = [x for x in Path(exportdir).glob("*") if x.is_dir()]

if exported_file_count == len(interim_ids_current_site):
    print("Correct number of exported files")
else:
    print("Unexpected number of exported .db files")
    print(f"\tFound: {exported_file_count}")
    print(f"\tExpected: {len(interim_ids_current_site)}")

if exported_folder_count == len(interim_ids_current_site):
    print("Correct number of exported folders")
else:
    print("Unexpected number of exported folders")
    print(f"\tFound: {exported_folder_count}")
    print(f"\tExpected: {len(interim_ids_current_site)}")
```

1.2.6 Step 6 - Identify files to extract from export

The Python code will recursively scan the exported files to identify and copy any files that meet any of the following sets of criteria:

Lesion Data JSONs - Named *lesion_data_*.json* - In a folder named *analysis* - At least 1MB - Last JSON in folder when sorted alphabetically (effectively the highest version number)

User modifications file - Named *user_modifications.ini* - In a folder named “analysis”.

TOM file - Named **.tom* - Not in a folder with the text “Tracked”, “lms”, or “template” in the folder name.

Raw image files * Any file called “a19B.CR2”

DexiData - Named **.json* - In a folder named *DexiData**

The next cell will scan the exported data for files that meet the previously described criteria. Matching files will be saved in a list contained in a CSV file. **This might take a long time!**. Set the following variable in the next cell, and then run the cell:

- **file_list_csv** - the full path of where to save the file list CSV file

```
[ ]: file_list_csv: Path = Path(".")

def find_matching_files(root_path: Path) -> list:

    MIN_JSON_SIZE: int = 1 * 1024 * 1024 # 1MB

    matching_files: list = []
```

```

for dirpath, dirnames, filenames in os.walk(root_path):
    path: Path = Path(dirpath)

    # Convert filenames to Path objects for easy filtering
    files: list[Path] = [Path(f) for f in filenames]

    # Condition 1: Max JSON in 'analysis' folder, >=1MB
    if path.name == "analysis":
        json_files: list[Path] = [
            path / f for f in files
            if f.suffix == ".json" and Path(path, f).stat().st_size >=
↳MIN_JSON_SIZE
        ]
        if json_files:
            max_json: Path = max(json_files, key=lambda f: f.name)
            #matching_files.append(max_json)
            matching_files.append(
                {
                    "path": max_json,
                    "category": "lesion_data_json",
                    "size_bytes": max_json.stat().st_size,
                    "timestamp": max_json.parent.parent.name
                }
            )

    # Condition 3: 'user_modifications.ini' in 'analysis' folder
    if path.name == "analysis":
        if "user_modifications.ini" in filenames:
            #matching_files.append(Path(dirpath) / 'user_modifications.ini')
            matching_files.append(
                {
                    "path": path / "user_modifications.ini",
                    "category": "user_mods",
                    "size_bytes": Path(path, "user_modifications.ini").
↳stat().st_size,
                    "timestamp": path.parent.name
                }
            )

    # Condition 4: *.tom where parent != 'Tracked'
    for f in files:
        if "template" not in f.name and f.suffix.lower() == ".tom" and path.
↳name != "Tracked" and path.name != "lms":
            #matching_files.append(Path(dirpath) / f)
            matching_files.append(
                {
                    "path": path / f,

```

```

        "category": "tom_file",
        "size_bytes": Path(path, f).stat().st_size,
        "timestamp": path.name
    }
)

# Condition 5: 'a19B.CR2'
if 'a19B.CR2' in filenames:
    #matching_files.append(Path(dirpath) / 'a19B.CR2')
    matching_files.append(
        {
            "path": path / "a19B.CR2",
            "category": "image_back",
            "size_bytes": Path(path, "a19B.CR2").stat().st_size,
            "timestamp": path.name
        }
    )

# Condition 6: any JSON file in folder containing 'DexiData'
if any("DexiData" in part for part in path.parts):
    for f in files:
        if f.suffix == '.json':
            #matching_files.append(Path(dirpath) / f)
            matching_files.append(
                {
                    "path": path / f,
                    "category": "dexi_json",
                    "size_bytes": Path(path, f).stat().st_size,
                    "timestamp": path.parent.name
                }
            )

    )

return matching_files

file_list: list[dict[str, any]] = find_matching_files(exportdir)
file_list_df: pd.DataFrame = pd.DataFrame(file_list)
file_list_df.to_csv(file_list_csv)

print(
    f"Results:\n"
    f"\tFile count: {len(file_list_df)}\n"
    f"\tTotal filesize: {round(file_list_df.size_bytes.sum() / (1024**3), 2)}\n"
    f"\tGB"
)

```

1.2.7 Step 7 - Copy the identified files

The next cell will copy the files identified in Step 6 into a new location. This may take some time, however a progress bar will display during the operation. Before running the cell, you must set the following variable:

- `file_copy_destination` - the destination folder to which the files will be copied.

```
[ ]: file_copy_destination: Path = Path("...")

for idx, row in tqdm(file_list_df.iterrows(), shape=file_list_df.shape[0]):
    src: Path = row["path"]
    dst: Path = Path(file_copy_destination, *src.parts[1:])
    shutil.copyfile(src, dst)
```

1.2.8 Data processing

todo